

Spring Boot Messaging

In diesem Modul

- Warum Messaging? Modelle (Queue vs. Topic) und typische Use Cases
- JMS mit Spring: Producer/Listener, Message Converter, Transaktionen & Idempotenz
- AMQP/RabbitMQ: Exchanges/Bindings, Producer/Consumer, DLX/DLQ
- Kafka: Topics/Partitionen, Producer/Consumer Groups, Serdes, Fehlerbehandlung
- Reliability: Acks, Confirms, Retry/Backoff, Dead Letter
- Event-Driven Architectures: Domain Events, Sagas
- Übungen

Wiederholung: Warum Messaging?

- **Asynchrone Kommunikation:** Sender und Empfänger müssen nicht gleichzeitig verfügbar sein.
- **Entkopplung:** Services kennen sich nicht direkt, kommunizieren über Nachrichtenkanäle.
- **Resilienz:** Bei Ausfall eines Empfängers gehen Nachrichten nicht verloren (werden gepuffert).
- **Skalierbarkeit:** Einfaches Hinzufügen weiterer Consumer für erhöhten Durchsatz.
- **Event-Driven Architectures (EDA):** Basis für moderne verteilte Systeme.

Wiederholung: Messaging-Modelle

1. Point-to-Point (Queues)

- Nachricht wird an eine Queue gesendet.
- **Nur ein Consumer** empfängt und verarbeitet die Nachricht.
- Ideal für Work-Distribution und Lastverteilung.

2. Publish/Subscribe (Topics / Exchanges)

- Nachricht wird an ein Topic (oder Exchange) gesendet.
- **Alle Subscriber**, die das Topic abonniert haben, erhalten eine Kopie der Nachricht.
- Ideal für Benachrichtigungen und Event-Broadcasting.

Wann nutzt man Messaging?

- **Bestellabwicklung:** Bestellung aufgeben (async zu Payment, Shipping, Notification).
- **Benachrichtigungen:** E-Mails, SMS, Push-Nachrichten versenden.
- **Daten-Integration:** Synchronisierung von Daten zwischen Systemen.
- **Batch-Verarbeitung:** Lange laufende Aufgaben auslagern.
- **Circuit Breaker / Bulkhead Pattern:** Erhöhung der Systemstabilität.

JMS (Java Message Service) in Spring Boot

Die JMS Spezifikation

- JMS ist eine Standard-API für Messaging in Java.
- Definiert gemeinsame Konzepte: `ConnectionFactory`, `Connection`, `Session`, `MessageProducer`, `MessageConsumer`, `Queue`, `Topic`, `Message`.
- Unabhängig vom konkreten Messaging-Anbieter (ActiveMQ, IBM MQ, TIBCO EMS).

Spring JMS mit ActiveMQ (Beispiel-Broker)

Dependency: `spring-boot-starter-activemq`

Nachrichten Senden (`JmsTemplate`)

```
@Service
public class OrderProducer {

    private final JmsTemplate jmsTemplate;

    public OrderProducer(JmsTemplate jmsTemplate) {
        this.jmsTemplate = jmsTemplate;
    }

    public void sendOrder(Order order) {
        System.out.println("Sending order: " + order.getId());
        // Konvertiert das Objekt automatisch in eine JMS Message (z.B. TextMessage, ObjectMessage)
        jmsTemplate.convertAndSend("orderQueue", order);
    }

    public void sendOrderStatus(String status) {
        System.out.println("Sending status update: " + status);
        jmsTemplate.convertAndSend("orderTopic", status);
    }
}
```


Nachrichten Empfangen (@JmsListener)

```
@Component
public class OrderConsumer {

    @JmsListener(destination = "orderQueue")
    public void receiveOrder(Order order) {
        System.out.println("Received order: " + order.getId() + " - Processing...");
        // Hier: Logik zur Verarbeitung der Bestellung
    }

    @JmsListener(destination = "orderTopic", containerFactory = "jmsTopicFactory")
    public void receiveOrderStatus(String status) {
        System.out.println("Received status update on topic: " + status);
    }
}
```

containerFactory = "jmsTopicFactory"

Notwendig, wenn man ein Topic statt einer Queue verwendet und Spring nicht die Standardeinstellungen anwenden soll. Konfiguration in `application.yml`:

```
spring:
  jms:
    pub-sub-domain: true # Aktiviert den Pub/Sub-Modus für den JmsTemplate
```

Message Converters

- Wandeln Java-Objekte in `javax.jms.Message` und umgekehrt.
- Spring Boot konfiguriert standardmäßig den `MappingJackson2MessageConverter` für JSON.

```
@Configuration
public class JmsConfig {

    @Bean
    public MessageConverter jacksonJmsMessageConverter() {
        MappingJackson2MessageConverter converter = new MappingJackson2MessageConverter();
        converter.setTargetType(MessageType.TEXT); // JSON als TextMessage
        converter.setTypeIdPropertyName("_type"); // Typ-Information für Deserialisierung
        return converter;
    }
}
```

Transaktionen mit JMS

- **Lokale Transaktionen:** `JmsTransactionManager`. Nachrichten werden erst bei Commit gesendet, Rollback bei Fehlern.
 - Für Single-Resource-Transaktionen (z.B. nur JMS).
- **Verteilte Transaktionen (XA / JTA):** Wenn die JMS-Operation und eine Datenbank-Operation atomar sein müssen (entweder beides commit oder beides rollback).
 - Benötigt einen XA-kompatiblen JMS-Broker und einen JTA-Transaktionsmanager (z.B. Atomikos, Narayana).
 - Hohe Komplexität und Performance-Einbußen.
 - Alternative: Saga Pattern (Modul 3).

Idempotenz

- Wichtig, da Nachrichten in verteilten Systemen **mehrfach zugestellt** werden können ("at-least-once" Delivery).
- Eine Operation ist idempotent, wenn sie mehrmals ausgeführt werden kann, ohne zusätzliche Seiteneffekte zu erzeugen.
- **Strategien:**
 - Eindeutige Message-ID verfolgen.
 - Status-Management (nur bei Status "pending" verarbeiten).
 - Database Unique Constraints.

AMQP in Spring Boot

Das AMQP-Modell

- Ein offener Standard für Messaging.
- Flexibler und mächtiger als JMS, da das Routing-Modell entkoppelt ist.
- Wichtige Konzepte:
 - **Producer:** Sendet Nachrichten.
 - **Exchange:** Empfängt Nachrichten vom Producer und leitet sie an Queues weiter.
 - **Binding:** Eine Regel, die eine Queue an einen Exchange bindet.
 - **Queue:** Speichert Nachrichten, bis sie von einem Consumer abgeholt werden.
 - **Consumer:** Empfängt Nachrichten von einer Queue.

Exchange Types

1. Direct Exchange:

- Nachricht geht an Queues, deren Binding Key *exakt* dem Routing Key der Nachricht entspricht.
- Ideal für 1:1 oder 1:N Weiterleitung, wenn der Key bekannt ist.

2. Topic Exchange:

- Nachricht geht an Queues, deren Binding Key einem Wildcard-Muster des Routing Keys entspricht.
- `*` : Ersetzt genau ein Wort.
- `#` : Ersetzt null oder mehr Worte.
- Ideal für Pub/Sub mit feingranularer Filterung.

3. Fanout Exchange:

- Nachricht geht an *alle* Queues, die an diesen Exchange gebunden sind (Routing Key wird ignoriert).
- Ideal für Broadcasting.

4. Headers Exchange:

- Leitet basierend auf den Headern der Nachricht weiter (seltener verwendet).

Spring AMQP mit RabbitMQ

Dependency

```
spring-boot-starter-amqp
```

Konfiguration (Minimal)

```
spring:  
  rabbitmq:  
    host: localhost  
    port: 5672  
    username: guest  
    password: guest
```

Nachrichten Senden (RabbitTemplate)

```
@Service
public class MessageProducer {

    private final RabbitTemplate rabbitTemplate;

    public MessageProducer(RabbitTemplate rabbitTemplate) {
        this.rabbitTemplate = rabbitTemplate;
    }

    public void sendMessage(String exchange, String routingKey, Object message) {
        System.out.println("Sending to exchange " + exchange + " with routingKey " + routingKey);
        rabbitTemplate.convertAndSend(exchange, routingKey, message);
    }

    public void publishEvent(Object event) {
        // Beispiel: Fanout Exchange für Events
        rabbitTemplate.convertAndSend("events.fanout", "", event);
    }
}
```

Nachrichten Empfangen (@RabbitListener)

```
@Component
public class MessageConsumer {

    @RabbitListener(queues = "myQueue")
    public void receiveMessage(String message) {
        System.out.println("Received from myQueue: " + message);
    }

    @RabbitListener(queues = "logQueue")
    public void receiveLog(LogMessage log) {
        System.out.println("Received Log: " + log.getLevel() + " - " + log.getContent());
        // Hier könnte eine manuelle Acknowledge-Logik stattfinden
        // channel.basicAck(deliveryTag, false);
    }
}
```

Automatische Erstellung von Exchanges, Queues und Bindings

Spring AMQP kann diese bei Anwendungsstart automatisch erstellen.

```
@Configuration
public class RabbitConfig {

    @Bean
    public Queue myQueue() {
        return new Queue("myQueue", true); // Name, durable
    }

    @Bean
    public TopicExchange topicExchange() {
        return new TopicExchange("logExchange");
    }

    @Bean
    public Binding binding(Queue myQueue, TopicExchange topicExchange) {
        return BindingBuilder.bind(myQueue).to(topicExchange).with("*.critical.#"); // Routing Key Muster
    }

    @Bean // Fanout Exchange für Events
    public FanoutExchange eventsFanoutExchange() {
        return new FanoutExchange("events.fanout");
    }

}
```

Message Converters

Analog zu JMS wandeln sie Java-Objekte in `Message` und umgekehrt.
Standard ist auch hier `MappingJackson2MessageConverter` .

Reliable Messaging & Dead-Letter Queues

Publisher Confirms & Returns

- **Confirms:** Der Broker bestätigt dem Publisher, dass er die Nachricht erhalten hat.
- **Returns:** Der Broker benachrichtigt den Publisher, wenn eine Nachricht an keinen Consumer zugestellt werden konnte.
- Wichtig für "at-least-once" oder "exactly-once" Semantik (mit Idempotenz).

```
// Konfiguration im RabbitTemplate  
// rabbitTemplate.setConfirmCallback(...)  
// rabbitTemplate.setReturnCallback(...)
```


Consumer Acknowledgements

Wie ein Consumer dem Broker mitteilt, dass die Nachricht erfolgreich verarbeitet wurde.

1. **AUTO (Default in Spring Boot)**: Automatisch bei erfolgreicher Methodenausführung.
2. **MANUAL** : Consumer muss explizit `channel.basicAck()` oder `channel.basicNack()` aufrufen.
 - Wichtig bei komplexer Verarbeitung, die fehlschlagen könnte.

Dead-Letter Exchanges (DLX)

- Nachrichten, die nicht verarbeitet werden können (z.B. wegen Exceptions, NACKs, TTL-Ablauf), werden an einen speziellen Exchange (DLX) gesendet.
- Von dort können sie in eine **Dead-Letter Queue (DLQ)** geleitet werden.
- Wichtig für Fehlerbehandlung und Auditing.

Konfiguration einer Queue mit DLX

```
@Bean
public Queue processingQueue() {
    return QueueBuilder.durable("processingQueue")
        .withArgument("x-dead-letter-exchange", "dlxExchange")
        .withArgument("x-dead-letter-routing-key", "processing.dlq")
        .build();
}

@Bean
public DirectExchange dlxExchange() {
    return new DirectExchange("dlxExchange");
}

@Bean
public Queue dlq() {
    return new Queue("processing.dlq");
}

@Bean
public Binding dlqBinding(Queue dlq, DirectExchange dlxExchange) {
    return BindingBuilder.bind(dlq).to(dlxExchange).with("processing.dlq");
}
```

Spring Boot und Kafka

Die "Log-zentrierte" Architektur

- Kafka ist ein **verteiltes Streaming-Plattform**, kein klassischer Message Broker.
- Speichert Nachrichten in einem **Commit Log** (Topic).
- Nachrichten werden nicht "konsumiert" und gelöscht, sondern bleiben für eine konfigurierbare Zeit erhalten.

Hauptunterschiede

Feature	JMS/AMQP (Queues)	Kafka (Topics)
Nachrichten-Modell	Löschen nach Konsum	Persistenz (log-basiert)
Konsum-Muster	Point-to-Point (Queue), Pub/Sub (Topic)	Pub/Sub (Topic), Consumer Groups
Skalierung Consumer	Horizontale Skalierung (mehr Listener)	Horizontale Skalierung über Partitionen
Reihenfolge	Garantiert in einer Queue	Garantiert <i>innerhalb einer Partition</i>
Anwendungsfälle	Task Queues, Notifications	Event Sourcing, Stream Processing

Kafka Kernkonzepte

- **Broker:** Server, der Topics verwaltet.
- **Topic:** Logischer Kanal für Nachrichten.
- **Partition:** Ein Topic ist in Partitionen unterteilt (Skalierung, Parallelisierung).
- **Producer:** Schreibt Nachrichten in Topics/Partitionen.
- **Consumer:** Liest Nachrichten aus Topics/Partitionen.
- **Consumer Group:** Eine Gruppe von Consumern, die gemeinsam ein Topic verarbeitet. Jede Nachricht in einer Partition wird nur an *einen* Consumer *innerhalb der Gruppe* zugestellt.

Apache Kafka in Spring

Dependency

```
spring-kafka
```

Konfiguration (Minimal)

```
spring:
  kafka:
    bootstrap-servers: localhost:9092 # Adressen der Kafka Broker
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: my-service-group
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      auto-offset-reset: latest # Wo starten, wenn keine Offset gefunden
```


Nachrichten Senden (KafkaTemplate)

```
@Service
public class EventProducer {

    private final KafkaTemplate<String, Object> kafkaTemplate;

    public EventProducer(KafkaTemplate<String, Object> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publishUserCreatedEvent(UserCreatedEvent event) {
        System.out.println("Publishing UserCreatedEvent: " + event.getUserId());
        // Key wird für Partitioning genutzt (alle Nachrichten mit gleichem Key landen in gleicher Partition)
        kafkaTemplate.send("user-events-topic", event.getUserId().toString(), event);
    }
}
```

Nachrichten Empfangen (@KafkaListener)

```
@Component
public class UserEventListener {

    @KafkaListener(topics = "user-events-topic", groupId = "user-processor-group")
    public void listen(UserCreatedEvent event, @Header(KafkaHeaders.RECEIVED_PARTITION) int partition) {
        System.out.println("Received UserCreatedEvent for user " + event.getUserId() +
                           " from partition " + partition);
        // ... Logik zur Verarbeitung des Events
    }
}
```

Serde (Serializer/Deserializer)

- Kafka Nachrichten sind Byte-Arrays.
- Producer muss Objekte serialisieren, Consumer deserialisieren.
- Spring Kafka bietet `JsonSerializer` / `JsonDeserializer` für JSON.

Fehlerbehandlung

- **Consumer Group Offsets:** Kafka merkt sich pro Consumer Group den letzten verarbeiteten Offset.
- **Retry-Mechanismen:** Bei Fehlern die Nachricht erneut versuchen.
- **Dead-Letter Topics (DLT):** Nachrichten, die dauerhaft nicht verarbeitet werden können, an ein spezielles Error-Topic senden.
 - Spring Kafka bietet `DeadLetterPublishingRecoverer`.

Event-Driven Architectures (EDA)

Das Konzept

- Systeme kommunizieren primär durch das Publizieren und Konsumieren von Events.
- Starke Entkopplung: Services kennen sich nicht, reagieren nur auf relevante Events.
- Grundlage für Microservices, Reaktivität und Skalierbarkeit.

Domain Events

- Repräsentieren etwas, das in der Domäne passiert ist (z.B. `OrderPlacedEvent`, `PaymentFailedEvent`).
- Sind "Fakten" über vergangene Ereignisse.
- Werden von einem Service publiziert und von anderen Services (Consumer) verarbeitet.

Saga Pattern

- Ein Muster, um verteilte Transaktionen in EDAs zu handhaben (siehe Modul 3).
- Eine Abfolge von lokalen Transaktionen, orchestriert durch Events.
- Bei Fehlern werden Kompensations-Events ausgelöst.

Event Sourcing

- Statt nur den aktuellen Zustand zu speichern, werden *alle Events* gespeichert, die zu diesem Zustand geführt haben.
- Der aktuelle Zustand wird durch "Replay" aller Events rekonstruiert.
- Vorteile: Auditability, Time-Travel Debugging, einfaches Hinzufügen neuer View-Modelle.
- Kafka ist eine ausgezeichnete Plattform für Event Sourcing.